



Department of
COMPUTER SCIENCE AND ENGINEERING

Blood Donation Management System

Submitted By

Student Name	Student ID
JIHADUL ISLAM SIAM	2243081047
SAJJADUL ISLAM NOMAN	2243081059
ASADUJJAMAN NAYEEM	2243081055
NAFIS KHAN	2243081070
MD. ABDUL KARIM	2243081080

MINI LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course **CSE0613202:**
OBJECT ORIENTED PROGRAMMING LAB in the Computer
Science and Engineering Department.

Submitted To:

NAHRIN JANNAT

Lecturer

Department of Computer Science and Engineering Uttara University

Project Objective:

The objective of this Object-Oriented Programming (OOP) course project is to develop a console-based Blood Donation Management System using C# that effectively demonstrates the practical application of object-oriented design principles. The project is structured to model real-world components of a blood donation process by representing donors, consumers, and blood requests as distinct classes with clearly defined responsibilities. Through this approach, the system emphasizes the use of encapsulation to protect sensitive data, inheritance to promote code reuse, abstraction to define common behaviors, and polymorphism to handle different types of users and requests efficiently.

In addition, the project aims to create a logically organized and user-friendly system that supports user registration, secure login, donor profile management, blood request submission, and request acceptance based on location and blood group compatibility. By dividing the application into multiple management modules and interactive menus, the project highlights proper class interaction, separation of concerns, and maintainable code structure. Overall, this project is intended to strengthen our understanding of OOP concepts.

Project Review:

This project successfully demonstrates the practical implementation of Object-Oriented Programming concepts through the development of a console-based Blood Donation Management System. The system effectively models real-world entities such as donors, consumers, and blood requests using classes, inheritance, and abstraction. Core OOP principles including encapsulation, inheritance, abstraction, and limited polymorphism have been applied to ensure structured design and code reusability.

The project provides essential functionalities such as user registration, authentication, donor profile management, blood request creation, and request acceptance based on location and blood group compatibility. The use of separate management classes for users and blood requests improves code organization and separation of responsibilities. Overall, the project fulfills its academic objective by offering a functional and well-structured application that aligns with OOP best practices and course requirements.

Limitations Faced During Project Development:

During the development of this Object-Oriented Programming (OOP) course project, several limitations were encountered. One of the major constraints was the inability to use a database system. As a result, all user and request data had to be stored in arrays of objects within the program. This approach limited the scalability of the system and caused all stored data to be lost once the application was closed, as there was no mechanism to save or retrieve data from previous executions.

Another limitation was the absence of any file-handling or data persistence feature. Since no external storage was used, previously registered users, donors, and blood requests could not be retained after the program terminated. This restricted the system to temporary, runtime-only data management.

Additionally, challenges were faced while applying object-oriented programming concepts such as encapsulation, inheritance, abstraction, and polymorphism. Designing appropriate class relationships, managing access modifiers, and correctly implementing abstract classes and inherited behaviors required careful planning and multiple revisions. These challenges increased development time and complexity but also provided valuable learning experience in understanding and applying OOP principles effectively within a real-world problem scenario.

Method:

NAFIS KHAN – 2243081070

1. Class and Object

The code is structured using multiple classes such as Person, Doner, Consumer, Request, BloodRequest, and ManageBloodRequest.

Each class represents a real-world entity in the blood donation system. Objects of these classes are created to store and manipulate data during program execution.

2. Encapsulation

Encapsulation is implemented to protect sensitive data and control access to class members.

Example:

- The password variable in the Person class is declared as private.
- A public property Password is used to access and modify the password securely.

This ensures data hiding and prevents unauthorized direct access to sensitive information.

3. Inheritance

Inheritance is used to promote code reusability and establish hierarchical relationships between classes.

Examples:

- Doner and Consumer classes inherit from the Person class.
- BloodRequest class inherits from the abstract class Request.

Through inheritance, child classes reuse properties of parent classes without duplicating code.

4. Abstraction

Abstraction is achieved using an abstract class.

Example:

- The Request class is declared as abstract.
- It defines common properties related to blood requests but cannot be instantiated directly.

This hides implementation details and ensures that only specific request types (such as BloodRequest) are created.

5. Polymorphism (Compile-Time)

Compile-time polymorphism is implemented using **constructor overloading**.

Example:

- The Doner class has multiple constructors with different parameter lists.

This allows creating donor objects in different ways depending on available information.

6. Static Class and Members

The ManageBloodRequest class is declared as static.

Purpose:

- To manage blood requests globally.
- To ensure a single shared instance of request data.

All methods and variables inside this class are static, enabling centralized control without object creation.

MD. ABDUL KARIM – 2243081080

1. Class and Object

The ManageUser class is used to manage donor and consumer information in the system. It works with objects of the Doner and Consumer classes by storing them in arrays. These objects represent real users of the blood donation system and are created and managed during program execution.

2. Encapsulation

Encapsulation is used to protect user data and control access to it.

Example:

The donor and consumer arrays (doners and consumers) and their index variables are declared as private, which prevents direct access from outside the class. Access to these data members is only possible through public methods such as AddDoner, GetDoner, AddConsumer, and GetConsumer, ensuring controlled and secure data handling.

3. Static Class and Members

The ManageUser class is declared as a static class.

Purpose:

The static class is used to manage all users globally within the application. Since all variables and methods are static, there is no need to create objects of the ManageUser class, and user data remains shared and consistent throughout the program.

4. Polymorphism (Compile-Time)

Compile-time polymorphism is implemented through method overloading.

Example:

The AddDoner method is overloaded with different parameter lists. This allows donor information to be added in multiple ways depending on whether address and blood group details are available or not.

JIHADUL ISLAM SIAM – 2243081047

1. Class and Object

The Program class acts as the main controller of the application. It coordinates user interaction, authentication, registration, and navigation between different system features. Objects of the Doner and Consumer classes are used to represent the currently logged-in users through the variables loggedInDoner and loggedInConsumer.

2. Encapsulation

Encapsulation is applied by accessing user data through methods instead of directly manipulating internal data.

Example:

User authentication and validation are handled using methods such as DonerPhnAvailable, ConsumerPhnAvailable, DnrCradintialsCorrect, and

ConsCradintialsCorrect from the ManageUser class. This ensures that sensitive user data, such as passwords, is accessed in a controlled manner and not directly from the Program class.

3. Static Methods and Variables

The methods and variables inside the Program class are declared as static.

Purpose:

Static variables such as loggedInDoner and loggedInConsumer store the current session's user information, while static methods like Authentication, Registration, LoginDoner, and LoginConsumer control the application flow. This allows the program to run without creating an instance of the Program class.

4. Association

Association is demonstrated through interaction between the Program class and other classes.

Example:

The Program class interacts with the ManageUser class to register users, validate login credentials, and retrieve Doner and Consumer objects. It also stores references to these objects during user sessions, showing a “uses-a” relationship between classes.

5. Control Flow and Exception Handling

The code uses conditional statements and try-catch blocks to control program execution and handle runtime errors. This helps maintain program stability and ensures smooth user interaction, supporting the overall object-oriented structure of the application.

SAJJADUL ISLAM NOMAN – 2243081059

1. Class and Object

This part of the code uses the Doner class object to represent the currently logged-in donor. The object loggedInDoner is used to access and display donor

information such as name, phone number, address, blood group, and last donation date. This object-based approach allows donor-specific data to be handled throughout the donor home section.

2. Encapsulation

Encapsulation is maintained by updating donor information through methods instead of direct data manipulation.

Example:

The EditDonerDetails method collects updated donor information and passes it to the ManageUser.EditDonerDetails method. This ensures that donor data is modified in a controlled way, keeping data management separate from user interface logic.

3. Static Methods and Variables

All methods in this section, such as DonerHome, DonerProfile, DonerBloodRequest, and donerAcceptedrequest, are declared as static.

Purpose:

Static methods allow these features to be accessed directly without creating an object of the Program class. The static variable loggedInDoner stores the active donor session and is shared across all these methods.

4. Association

Association is demonstrated through interaction between the Program class and other classes.

Example:

The donor home methods interact with the ManageUser class to update donor details and with the ManageBloodRequest class to view and accept blood requests. This shows a “uses-a” relationship where the Program class uses services provided by other classes.

5. Polymorphism

Polymorphism is indirectly used by passing the loggedInDoner object to methods such as ShowDonerRequest, Accept, and ShowDonerAcceptedRequest. These methods operate based on the donor object's data, allowing flexible behavior depending on the donor information provided.

ASADUJJAMAN NAYEEM – 2243081055

1. Class and Object

This section of the code uses the Consumer class object to represent the currently logged-in consumer. The object loggedInConsumer is used to access consumer information such as name and phone number and to perform consumer-specific operations like sending blood requests and viewing request history.

2. Encapsulation

Encapsulation is applied by handling consumer actions through methods rather than directly modifying data.

Example:

The consumer's information is accessed using the loggedInConsumer object, while request creation and retrieval are performed through methods of the ManageBloodRequest class. This ensures that request data and user data are managed in a controlled and secure manner.

3. Static Methods and Variables

All methods in this section, such as ConsumerHome, ConsumerProfile, SendBloodRequest, and ShowConsumerRequest, are declared as static.

Purpose:

Static methods allow consumer-related functionality to be executed without creating an object of the Program class. The static variable loggedInConsumer stores the active consumer session and is shared across all consumer-related methods.

4. Association

Association is demonstrated through interaction between the Program class and other classes.

Example:

The Program class interacts with the ManageUser class to search for donors and with the ManageBloodRequest class to send and display blood requests. This shows a “uses-a” relationship where the consumer home section relies on other classes to perform its operations.

5. Abstraction

Abstraction is applied by hiding the internal working of donor searching and request management.

Example:

Methods such as ShowPeople, AddRequest, and ShowConsumerRequest are used without exposing their internal logic. This allows the consumer home section to focus only on user interaction while complex processing is handled by separate classes.

OUTPUT:

Registration

Consumer Registration:

```
1. Login as Doner
2. Login as Consumer
3. Registration
Choose: 3
----- Registration -----
1. As Doner
2. As Consumer
Choose: 2
----- Consumer Registration -----
Enter phone: 018
Enter your name: jihad
Create password: 1234
Registration successfull!
----- Authentication -----
1. Login as Doner
2. Login as Consumer
3. Registration
Choose: 2
----- Login Consumer -----
Enter your phone number: 018
Enter your password: 1234
----- Home -----
Welcome jihad
1. Profile
2. Send Request
3. My Request
0. Logout
Choose: 1
----- Profile -----
Name: jihad
Phone: 018
0. Back to home
Choose: 0
----- Home -----
Welcome jihad
1. Profile
2. Send Request
```

Donor Registration:

```
----- Authentication -----
1. Login as Doner
2. Login as Consumer
3. Registration
Choose: 3
----- Registration -----
1. As Doner
2. As Consumer
Choose: 1
----- Doner Registration -----
Enter phone number: 017
Enter name: Noman
Create password: 1234
1. Add address now
2. Create account anyway
Choose: 2
Registration successfull!
----- Authentication -----
1. Login as Doner
2. Login as Consumer
3. Registration
Choose: 1
----- Login -----
Enter your phone number: 017
Enter your password: 1234
Welcome Home
----- Home -----
Welcome Noman
1. My profile
2. Blood request
3. Accepted Request
0. Logout
Choose: █
```

Donor section

Donor profile :

```
----- Login -----
Enter your phone number: 017
Enter your password: 1234
Welcome Home
----- Home -----
Welcome Noman
1. My profile
2. Blood request
3. Accepted Request
0. Logout
Choose: 1
----- Profile -----
Name: Noman
Pone: 017
Division: null
District: null
Thana: null
Blood group: null
Last blood donation date: null
1. Edit profile
0. Back to home
Choose: 1
Enter your division: Dhaka
Enter your district: Gazipur
Enter your Thana: Sadar
Enter your blood group: A+
Edit successfull
----- Profile -----
Name: Noman
Pone: 017
Division: Dhaka
District: Gazipur
Thana: Sadar
Blood group: A+
Last blood donation date: null
1. Edit profile
0. Back to home
Choose: █
```

```
----- Authentication -----
. Login as Doner
. Login as Consumer
. Registration
choose: 1
----- Login -----
enter your phone number: 017
enter your password: 1234
welcome Home
----- Home -----
welcome Noman
. My profile
. Blood request
. Accepted Request
. Logout
choose: 2
----- Blood Request -----
-----
request id: 1
consumer name: jihad
blood Group: A+
quantity: 1
date & Time: 12-15-2025 18:00
address: Dhaka, Gazipur, Sadar, popular
-----
. Accept any
. Back to home
choose: 1
enter request id: 1
accepted request id: 1
----- Blood Request -----
-----
no request available now
. Accept any
. Back to home
choose: █
```

Consumer section

Consumer profile :

```
----- Authentication -----
1. Login as Doner
2. Login as Consumer
3. Registration
Choose: 2
----- Login Consumer -----
Enter your phone number: 018
Enter your password: 1234
----- Home -----
Welcome jihad
1. Profile
2. Send Request
3. My Request
4. Logout
Choose: 3
----- My request -----
-----
Blood Group: A+
Quantity: 1
Date & Time: 12-15-2025 18:00
Address: Dhaka, Gazipur, Sadar, popular
Accepted by:
Name: Noman
Phone: 017
Address: Dhaka, Gazipur, Sadar
-----
4. Back to home
Choose: █
```



```
----- Home -----
Welcome jihad
1. Profile
2. Send Request
3. My Request
0. Logout
Choose: 2
----- Send Request -----
Enter division: Dhaka
Enter district: Gazipur
Enter Thana: Sadar
Enter hospital: popular
Enter blood group: A+
Blood quantity: 1
Date (MM-DD-YYYY): 12-15-2025
Time (24HH:MM): 18:00
Search people?(Y/n): y
-----
Name: Noman
Blood group: A+
Address: Dhaka, Gazipur, Sadar
-----
Send request to all?(Y/n)y
Request send successfully
----- Home -----
Welcome jihad
1. Profile
2. Send Request
3. My Request
0. Logout
Choose: 3
----- My request -----
-----
Blood Group: A+
Quantity: 1
Date & Time: 12-15-2025 18:00
Address: Dhaka, Gazipur, Sadar, popular
```


Discussion

This project demonstrates the practical application of Object-Oriented Programming (OOP) concepts through the development of a console-based Blood Donation Management System. The system was designed to simulate a real-life scenario where donors and consumers interact through structured software components. By dividing the application into multiple classes such as users, donors, consumers, and blood requests, the project emphasizes modular design and clear separation of responsibilities.

Throughout the development process, significant focus was placed on applying OOP principles including encapsulation, inheritance, abstraction, and polymorphism. While implementing these concepts posed certain challenges, they helped improve understanding of class relationships, data protection, and code reusability. The use of static management classes allowed centralized control of users and requests, simplifying program flow and reducing complexity.

However, due to project constraints such as the absence of database and file storage, the system operates entirely in memory, limiting its persistence and scalability. Despite these limitations, the project successfully meets its academic goals by providing a functional system that reflects real-world logic and reinforces core OOP concepts.